

SLT-Mobitel AI Chatbot - Software Documentation

Document Version 1.3.2

Date: Oct 27, 2025

Prepared by:

Thinula Harischandra (2899)

Randil Jayasinghe (2907)

Mevinu Gunaratna (2898)

Keshara Gunathilaka (2901)



Table of Contents

<i>SLT-Mobitel AI Chatbot - Software Documentation</i>	1
1. Introduction	4
1.1 Project Overview	4
1.2 Purpose and Goals	4
1.3 Scope	4
1.5 Glossary of Terms.....	5
2. System Requirements	6
2.1 Hardware Requirements	6
2.2 Software Requirements	6
2.3 Network Requirements	6
3. Design and Architecture	7
3.1 Architectural Overview (RAG Model).....	7
3.2 Component Breakdown.....	8
3.3 Technology Stack.....	8
3.4 Data Flow Diagram.....	9
3.5 Data Models	10
4. LLM Interaction and Logic	11
4.1 Dual LLM Strategy (Ollama vs. Gemini).....	11
4.2 Initializing the LLM Connection.....	11
4.3 Prompt Engineering for RAG.....	12
4.4 Code Walkthrough: <code>query_llm</code>	13
5. Milvus Integration (Knowledge Base)	15
5.1 Why Milvus?	15
5.2 Installation and Configuration.....	15
5.3 Indexing: Inserting Data into Milvus.....	16
5.4 Retrieval: Querying Milvus for Context	16
6. In-Depth Code Explanation	17
6.1 Part 1: Data Ingestion Pipeline (The Scrapers)	17
6.2 Part 2: Core Chatbot Logic & User Interface	20

7. Installation and Configuration Guide.....	24
7.1 Prerequisites	24
7.2 requirements.txt.....	24
7.3 Step-by-Step Installation	24
7.4 Data Preparation (Crucial First Step)	25
7.5 Configuration	25
8. User Guide.....	26
8.1 Launching the Application	26
8.2 Interacting with the Chatbot	26
8.3 Example Conversations.....	26
9. API Reference	27
9.1 POST /chat	27
9.2 POST /location.....	27
10. Testing Strategy.....	28
11. Deployment.....	28
12. Security Considerations	28
13. Maintenance and Operations.....	29
14. Troubleshooting.....	29
15. Future Work and Improvements	29

1. Introduction

1.1 Project Overview

This document provides a comprehensive technical guide to the SLT-Mobitel AI Chatbot, an intelligent assistant designed to provide accurate and context-aware answers to user queries about Sri Lanka Telecom's services.

The application is built upon a modern Retrieval-Augmented Generation (RAG) architecture. This version uses a robust, decoupled data pipeline, storing versioned content in a **PostgreSQL** database, which is then indexed into a **Milvus** vector store. This ensures that responses are grounded in factual, up-to-date information.

1.2 Purpose and Goals

The primary purpose of this project is to reduce the burden on human customer service agents by automating responses to common user queries.

The key goals are:

- **Accuracy:** Provide answers based on the most current information available on the SLT website.
- **Availability:** Offer 24/7 support to customers.
- **Efficiency:** Deliver instant responses to frequently asked questions.
- **Flexibility:** Allow for deployment with either local, privacy-focused LLMs (Ollama) or powerful cloud-based LLMs (Groq).
- **Scalability:** Utilize a production-grade infrastructure (PostgreSQL, Milvus, FastAPI) capable of handling large volumes of data and user traffic.

1.3 Scope

In-Scope:

- Answering questions about SLT broadband packages, PEO TV, and new connection charges.
- Supporting both a local LLM (via Ollama) and a cloud-based LLM (Groq).
- Providing a web-based, responsive chat interface for user interaction.
- Implementing an advanced RAG pipeline with score boosting and a reranker for enhanced accuracy.

Out-of-Scope:

- Finding nearby SLT branch locations based on user-provided location or city name.
- User authentication and account management.
- Processing payments or modifying user subscriptions.
- Live-chat handoff to human agents.

1.4 Stakeholders

- **Users:** SLT-Mobitel customers seeking information.
- **Developers:** Team responsible for building and maintaining the application.
- **SLT Business Units:** Customer service and marketing departments who benefit from automated support.

1.5 Glossary of Terms

Term	Definition
Term	Definition
RAG	Retrieval-Augmented Generation. An AI architecture that retrieves factual information from a knowledge base to ground the responses of a Large Language Model (LLM).
LLM	Large Language Model. A deep learning model trained on vast amounts of text data, capable of understanding and generating human-like text.
Ollama	A tool for running open-source LLMs locally on a user's machine.
Vector Store	A specialized database (Milvus in this case) designed to store and search for information based on semantic meaning rather than exact keywords.
Embedding	A numerical representation (a vector) of text, capturing its semantic meaning.
PostgreSQL	A powerful, open-source object-relational database system used as the primary data store for scraped web content and metadata.
Milvus	A high-performance, open-source vector database designed to power embedding similarity search and AI applications.
FastAPI	A modern, high-performance Python web framework for building APIs.

2. System Requirements

2.1 Hardware Requirements

- **Data Pipeline Execution (Crawling, Vectorizing):**
 - **RAM:** 16 GB+ recommended for running web crawlers and embedding models efficiently.
 - **CPU:** Modern multi-core processor (e.g., Intel i7, AMD Ryzen 7).
- **Backend Application (FastAPI Server - Cloud LLM):**
 - Standard server/machine (4 GB RAM, 2-core CPU) is sufficient as the database, vector store, and LLM are external services.
- **Backend Application (FastAPI Server - Local LLM):**
 - **RAM:** 16 GB+ recommended for smooth operation (8 GB minimum).
 - **GPU:** (Optional but highly recommended for Ollama/LM Studio) NVIDIA GPU with 8 GB+ VRAM for acceptable performance. Without a GPU, response times will be significantly slower.

2.2 Software Requirements

- **OS:** Windows 10/11, macOS 12+, or a modern Linux distribution (e.g., Ubuntu 20.04+).
- **Core Engine:** Python 3.8 - 3.11.
- **Databases:** Access to a PostgreSQL instance.
- **Local LLM Server (Optional):** Ollama (latest version) or LM Studio.
- **Python Libraries:** See section 7.2 for the full `requirements.txt`.
- **Web Browser:** A modern browser with JavaScript enabled (e.g., Chrome, Firefox, Edge).

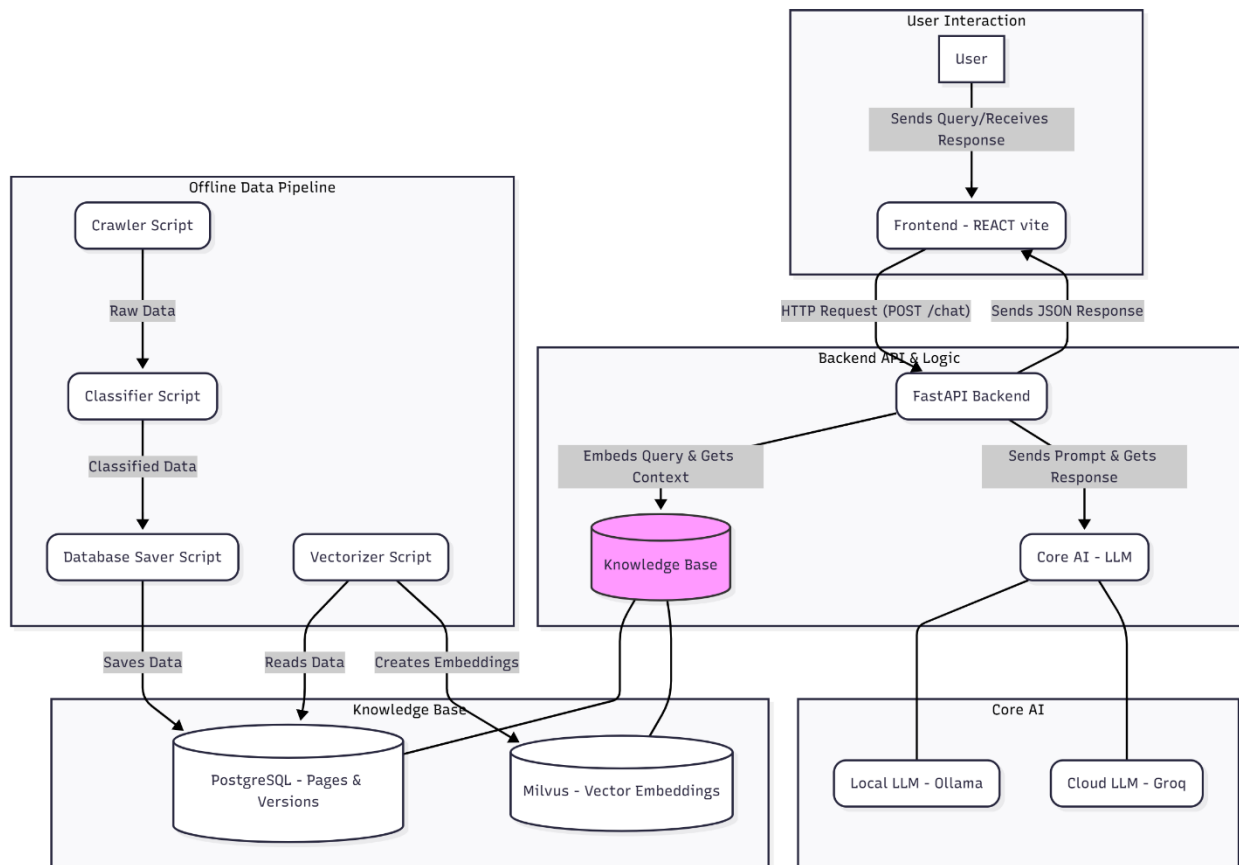
2.3 Network Requirements

- **Data Ingestion & Vectorization:** A stable internet connection is required to run the web scrapers and to push vectorized data to the cloud-based Milvus instance.
- **Runtime (Cloud LLM - Groq):** A stable internet connection is required for the FastAPI backend to connect to PostgreSQL, Milvus, and the Groq LLM API.
- **Runtime (Local LLM - Ollama):** No internet connection is required for the LLM generation step after the initial model download. An internet connection is still required to access the databases (unless they are also hosted locally).

3. Design and Architecture

3.1 Architectural Overview (RAG Model)

The chatbot operates on a Retrieval-Augmented Generation (RAG) model. This two-stage process significantly enhances the quality and accuracy of the LLM's responses.



The architecture functions in two distinct phases:

1. Offline Data Preparation (Indexing):

- The crawler scripts are run to scrape the SLT website.
- The scraped HTML is converted to clean markdown, and then further processed by an LLM for classification (category, tags, priority) and semantic cleaning.
- This processed content and metadata are stored in a versioned **PostgreSQL** database.
- A separate vectorization script reads updated content from PostgreSQL, converts it into numerical vector representations (embeddings), and stores these vectors in a **Milvus** vector store, creating a searchable knowledge library.

2. Online User Interaction (Retrieval & Generation):

- A user sends a query via the **Frontend**.
- The **FastAPI Backend** embeds the user's query.
- It performs a similarity search against the **Milvus** store to find the most relevant text chunks (the "context").
- These results are **boosted** based on priority/recency and then **reranked** for optimal relevance.
- A detailed prompt is constructed containing the user's query, conversation history, and the final retrieved context.
- This prompt is sent to the selected **LLM (Ollama or Groq)** to generate a response.

3.2 Component Breakdown

The system is composed of four main components:

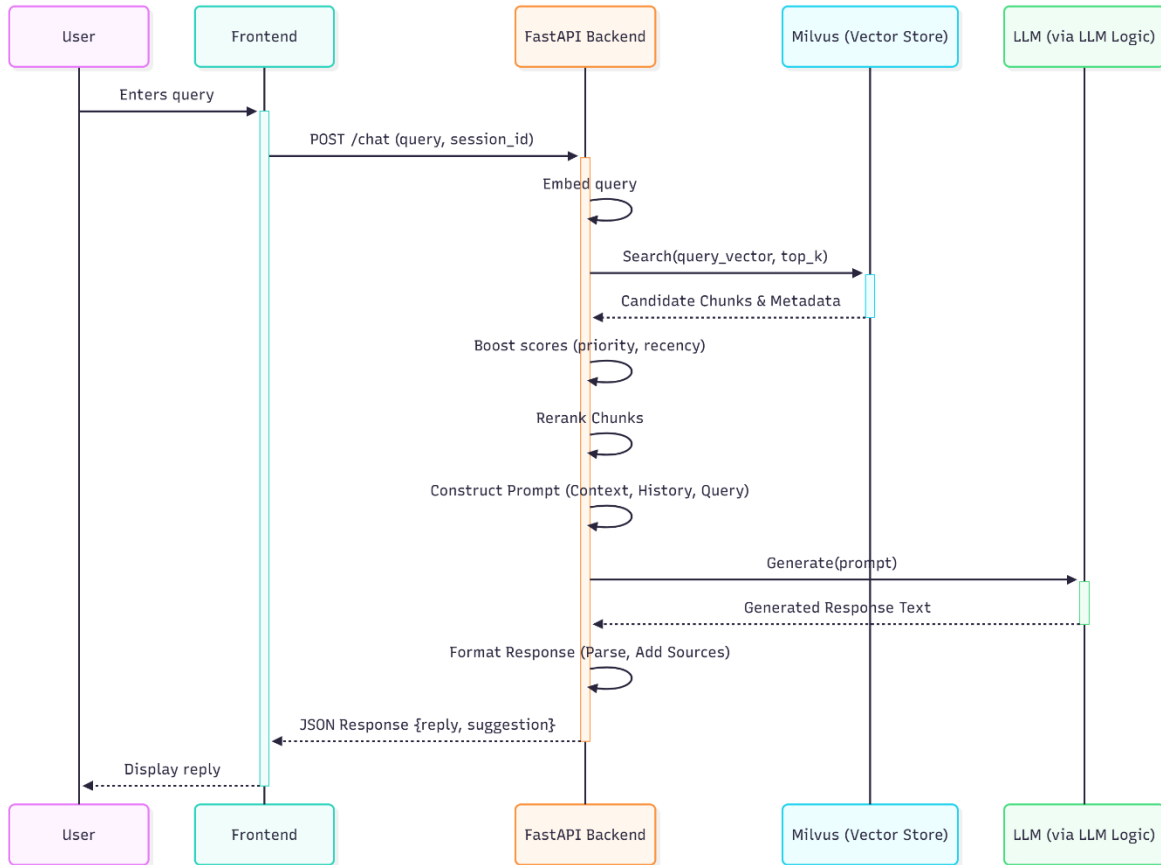
- **Data Ingestion Pipeline (Offline):** A set of Python scripts responsible for collecting, cleaning, classifying, and storing versioned web content in a PostgreSQL database.
- **Vectorization Pipeline (Offline):** A dedicated script that reads from PostgreSQL and populates the Milvus vector database with embeddings.
- **Knowledge Base:** A dual-database system comprising a **PostgreSQL** database for primary, versioned data and a **Milvus** collection for fast, semantic vector search.
- **Core Logic (Backend):** A **FastAPI** server that exposes an API for the frontend. It handles user queries by performing the RAG process, including retrieval, boosting, reranking, and generation.
- **Presentation Layer (Frontend):** An HTML file with embedded CSS and JavaScript that provides the user interface.

3.3 Technology Stack

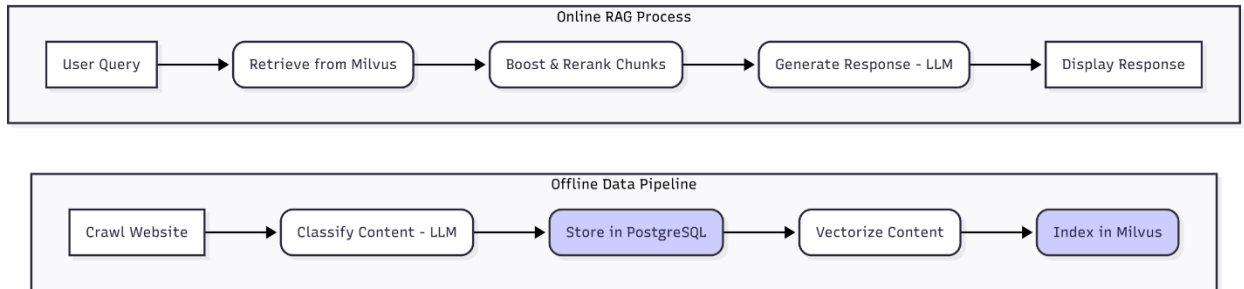
- **Backend:** Python 3.10+, **FastAPI**, Uvicorn
- **AI/ML:** **Sentence-Transformers**, **FlagEmbedding**, **LangChain** (for text splitters), Ollama, Groq API
- **Databases:** **PostgreSQL** (via asyncpg), **Milvus** (via pymilvus)
- **Web Scraping:** crawl4ai, BeautifulSoup4
- **Frontend:** HTML5, CSS3, JavaScript (ES6)
- **Utilities:** Pydantic

3.4 Data Flow Diagram

This diagram illustrates how data moves through the system from the user's initial query to the final response.



1. User enters a message in the frontend.
2. JavaScript sends a POST request to the /chat endpoint on the FastAPI server.
3. The backend receives the request and embeds the user's query.
4. It queries the **Milvus collection** for the top K similar chunks.
5. The results are **boosted** based on priority/recency and then **reranked** for relevance.
6. The backend constructs a prompt with these chunks and the query.
7. The prompt is sent to the configured LLM (Ollama or Groq).
8. The LLM generates a response.
9. The response is sent back to the frontend via the API.
10. JavaScript updates the UI with the bot's message.



3.5 Data Models

The system relies on structured schemas in PostgreSQL and Milvus. The previous `branches.json` file is no longer used.

PostgreSQL pages Table Schema This table stores the primary metadata and current state for each scraped URL.

```

CREATE TABLE IF NOT EXISTS pages (
  page_id UUID PRIMARY KEY,
  url TEXT UNIQUE NOT NULL,
  scraped_at TIMESTAMPTZ,
  title TEXT,
  meta_description TEXT,
  category TEXT,
  priority INT,
  tags TEXT[],
  checksum TEXT,
  content_md TEXT,
  is_vectorized BOOLEAN DEFAULT FALSE,
  current_version_id UUID,
  last_updated_at TIMESTAMPTZ
  -- Other metadata fields like etag, content_type may exist
);
  
```

PostgreSQL page_versions Table Schema This table stores the historical content for each page, allowing for version tracking.

```

CREATE TABLE IF NOT EXISTS page_versions (
  id UUID PRIMARY KEY,
  page_id UUID REFERENCES pages(page_id) ON DELETE CASCADE,
  scraped_at TIMESTAMPTZ,
  content TEXT, -- Stores the actual content (e.g., markdown)
  checksum TEXT, -- Hash of the content for comparison
  metadata JSONB, -- Additional version-specific metadata
  is_current BOOLEAN -- Flag indicating if this is the active version
);
  
```

Vector Store Metadata (Milvus Collection Schema) The vector store contains embeddings and associated metadata for efficient retrieval. This structure allows for searching and potentially filtering during the retrieval step.

- `id` (VARCHAR, Primary Key): Unique ID for each text chunk.
- `embedding` (FLOAT_VECTOR): The numerical vector representation of the text chunk.
- `page_id` (VARCHAR): Foreign key linking back to the pages table in PostgreSQL.
- `version_id` (VARCHAR): Identifier for the specific content version this chunk belongs to.
- `url` (VARCHAR): The source URL of the page.
- `category` (VARCHAR): The LLM-assigned category for the page.
- `priority` (DOUBLE): The assigned priority score for the page.
- `tags` (VARCHAR): A JSON string representation of the list of tags.
- `chunk_text` (VARCHAR): The actual text content of the chunk.
- `last_updated_at` (INT64): Timestamp used for recency boosting during retrieval.

4. LLM Interaction and Logic

This section details how the application initiates, configures, and communicates with the Large Language Models.

4.1 Dual LLM Strategy (Ollama vs. Gemini)

The application is architected to support multiple LLM providers to offer flexibility between privacy/offline use (Ollama, LM Studio) and high performance/capability (Groq, which replaces Gemini). This choice is controlled by a `DEFAULT_CONFIG` in the central LLM logic script and can be configured to use a local-first provider.

A key feature is the **fallback logic**. If a local provider like "ollama" is selected and the request fails (e.g., the local server is not running), the system will automatically "fall back" and retry the request using the cloud-based Groq provider. This ensures high availability for the user while still prioritizing local, private generation when possible.

4.2 Initializing the LLM Connection

Connection to the LLM is handled dynamically by the central `query_llm` function based on the selected configuration. There isn't a single initialization method run at startup for the LLM itself; instead, the connection or API call is made on demand.

- **For Cloud LLMs (Groq):** The system reads the `GROQ_API_KEY` from the environment variables and includes it as an Authorization header in the API request made by the `_query_groq` helper function. No persistent connection is maintained.

```
# Part of the logic for querying Groq
def _query_groq(prompt, config):
    headers = {
        "Authorization": f"Bearer {GROQ_API_KEY}", # API Key read from environment
        "Content-Type": "application/json",
    }
    payload = {
        "model": config["groq_model"],
        "messages": [{"role": "user", "content": prompt}],
        "temperature": config["temperature"],
    }
    res = requests.post(GROQ_API_URL, headers=headers, json=payload, timeout=30)
    # ... handle response ...
```

For Local LLMs (Ollama): The system connects to a predefined local URL (e.g., <http://127.0.0.1:11434/api/generate>) within the `_query_ollama` helper function each time a query is made. It does not require an API key but assumes the Ollama service is running and reachable at that address. A simple check like the one in the previous version might be added for robustness, but the core connection happens per-request.

```
# Part of the logic for querying Ollama
OLLAMA_URL = "http://127.0.0.1:11434/api/generate" # Predefined URL

def _query_ollama(prompt, config):
    payload = {
        "model": config["ollama_model"],
        "prompt": prompt,
        "stream": False
    }
    res = requests.post(OLLAMA_URL, json=payload, timeout=30) # Connects on demand
    # ... handle response ...
```

4.3 Prompt Engineering for RAG

The core of a successful RAG application is the quality of the prompt sent to the LLM. The prompt must be engineered to force the LLM to use the provided context and ignore its own pre-trained knowledge.

The prompt now has these main parts, constructed within the backend's `generate_answer` function:

- **System Instruction:** Defines the persona (expert SLT assistant) and core constraints (use *only* provided sources, cite sources, suggest follow-up).
- **Conversation History:** Includes recent turns to provide context for follow-up questions.
- **Context:** The factual information retrieved from Milvus and refined by the reranker, formatted clearly with source URLs.
- **User Query:** The current question asked by the user.
- **Format Instruction:** Explicitly tells the LLM the exact output structure (Answer: ... Follow-up question: ...).

```

# The updated prompt structure sent to the LLM
prompt = f"""
You are an expert assistant for Sri Lanka Telecom.
Answer the user's question using ONLY the information from the provided sources. If the answer is
not in the sources, say you don't know and suggest one clarifying question.

Guidelines:
- Be precise and helpful. Use simple language.
- Prefer the most recent and high-priority information implicitly.
- If listing options (plans, branches), present them clearly and concisely.
- Cite evidence using [Source N] inline where relevant.
- At the end, propose ONE natural follow-up question.

Conversation (last turns):
{history_text}

User question: {query}

Sources:
{context_text}

Format the output exactly as:
Answer: <your answer here with [Source N] citations>
Follow-up question: <one question>
"""

```

4.4 Code Walkthrough: `query_llm`

This function is the central point where the prompt is constructed and the LLM is called. In the new architecture, the prompt construction happens within the `generate_answer` function, which then calls the main `query_llm` routing function.

Step 1: Get History and Format Context

The function first retrieves the session history and formats the retrieved chunks (which were obtained from Milvus and reranked) into a clean, readable block, including source numbering and URLs.

```

# In the backend application script -> generate_answer function
def generate_answer(query: str, context_chunks: List[Dict[str, Any]], history: List[str]) -> Dict[str, str]:
    # Format context chunks with source numbers and URLs
    context_text, sources = _format_context_with_sources(context_chunks)
    # Format conversation history
    history_text = "\n".join([f"- {q}" for q in history[-6:]])

    # Helper function used above:
    # def _format_context_with_sources(chunks: List[Dict[str, Any]]) -> Tuple[str, List[str]]:
    #     lines = []
    #     sources = []
    #     for idx, ch in enumerate(chunks, start=1):
    #         url = ch.get("url") or ""
    #         txt = ch.get("chunk_text") or ""
    #         lines.append(f"[Source {idx}] URL: {url}\n{txt}") # Adds numbering and URL
    #         sources.append(url)
    #     return "\n\n".join(lines), sources

```

Step 2: Construct the System Prompt

The full prompt is assembled, explicitly instructing the LLM on its persona, constraints (use only provided sources, cite them), and the desired output format. This detailed instruction is crucial for RAG performance and preventing hallucinations.

```
# In the backend application script -> generate_answer function
# ... inside generate_answer, after formatting context and history ...
prompt = f"""
You are an expert assistant for Sri Lanka Telecom.
Answer the user's question using ONLY the information from the provided sources. If the answer is
not present in the sources, say you don't know and suggest one clarifying question.

Guidelines:
- Be precise and helpful. Use simple language.
- Prefer the most recent and high-priority information implicitly.
- If listing options (plans, branches), present them clearly and concisely.
- Cite evidence using [Source N] inline where relevant.
- At the end, propose ONE natural follow-up question.

Conversation (last turns):
{history_text}

User question: {query}

Sources:
{context_text}

Format the output exactly as:
Answer: <your answer here with [Source N] citations>
Follow-up question: <one question>
"""
```

Step 3: Call the Central LLM Router

Instead of directly calling a specific LLM API, the `generate_answer` function now calls the central `query_llm` function (detailed in Section 4.1). This central function handles the logic for choosing the provider (Ollama, Groq, etc.) based on configuration, making the actual API request, and managing fallbacks.

```
# In the backend application script -> generate_answer function
# ... inside generate_answer, after constructing the prompt ...

# Call the central LLM routing function
response = query_llm(prompt)

# ... (logic to parse the LLM's structured response) ...

# Append sources list for transparency
# ... (code to add source URLs to the final answer) ...

return {
    "reply": answer,
    "next_suggestion": suggestion
}

# The central query_llm function (simplified view, handles provider logic)
# def query_llm(prompt, config=None):
#     # ... (determines provider: ollama, groq, etc.) ...
#     # ... (calls appropriate helper: _query_ollama or _query_groq) ...
#     # ... (handles fallback logic) ...
#     # returns the LLM response string
```

5. Milvus Integration (Knowledge Base)

This section covers the installation, configuration, population (indexing), and querying (retrieval) of the vector store. The system has been migrated from ChromaDB to **Milvus** for enhanced scalability and performance.

5.1 Why Milvus?

The initial version of this project used ChromaDB. However, the system has been **migrated to Milvus (Zilliz Cloud)** to support the project's long-term goals.

Milvus was chosen as the new vector store for several key advantages:

- **Scalability:** Milvus is designed to handle billions of vectors, making it suitable for large-scale production environments.
- **Performance:** It offers highly optimized indexing (e.g., IVF_FLAT, HNSW) for fast and accurate similarity searches.
- **Cloud-Native:** It can be easily deployed and managed in a cloud environment (like Zilliz Cloud), separating the vector database from the application logic.
- **Reliability:** As a dedicated database, it provides features for data persistence, backup, and high availability.

5.2 Installation and Configuration

Milvus integration relies on the pymilvus library, which is included in the project's dependencies (requirements.txt). Configuration is handled via environment variables, not local installation files.

The key configuration variables required in the .env file are:

- **ZILLIZ_CLOUD_URI:** The connection endpoint for the Milvus instance.
- **ZILLIZ_CLOUD_API_KEY:** The authentication token for the service.
- **VECTOR_COLLECTION:** The name of the collection to use within Milvus.

Connection is established within the application code:

```
# In backend application and vectorization script initialization
from pymilvus import connections, Collection
# Read URI and TOKEN from environment variables
connections.connect(
    alias="default",
    uri=URI,
    token=TOKEN,
)
print(f"✅ Connected to Milvus Cloud")

# Load the specified collection
collection = Collection(VECTOR_COLLECTION)
collection.load()
print(f"✅ Collection '{VECTOR_COLLECTION}' loaded.")
```

5.3 Indexing: Inserting Data into Milvus

The indexing process has been updated for Milvus and is now part of a decoupled pipeline handled by a dedicated vectorization script.

Step 1: Fetch Data from PostgreSQL: The vectorization script queries the PostgreSQL pages table to find all active pages where `is_vectorized = FALSE`.

Step 2: Delete Existing Vectors: Before inserting new vectors for a page, the script first deletes any existing vectors associated with that `page_id` from Milvus. This ensures data consistency and removes outdated information.

```
# In vectorization script logic
def delete_vectors_by_page(collection: Collection, page_id: str):
    expr = f'page_id == "{page_id}"'
    collection.delete(expr)
    collection.flush() # Ensure deletion is committed
```

Step 3: Chunk and Create Embeddings: The page's markdown content is split into overlapping chunks using `RecursiveCharacterTextSplitter`. These chunks are then converted into vector embeddings in batches using a Sentence Transformer model (defined by the `EMBEDDING_MODEL` environment variable).

Step 4: Upsert Vectors to Milvus: The new vectors, along with their rich metadata (`page_id`, `url`, `category`, `priority`, `chunk text`, `last_updated_at`, etc.), are inserted into the Milvus collection using the `collection.insert()` method.

```
# In vectorization script logic
def upsert_vectors(collection: Collection, vectors: List[Dict[str, Any]]):
    # Prepare data in columnar format for Milvus insert
    ids = [v["id"] for v in vectors]
    embeddings = [v["embedding"] for v in vectors]
    page_ids = [v["page_id"] for v in vectors]
    # ... (prepare other metadata fields: version_ids, urls, etc.)

    entities = [
        ids,
        embeddings,
        page_ids,
        # ... (other metadata columns) ...
    ]
    collection.insert(entities)
    collection.flush() # Ensure data persistence
```

Step 5: Mark as Vectorized: Once the upsert is successful, the corresponding page in the PostgreSQL table is updated to `is_vectorized = TRUE`.

5.4 Retrieval: Querying Milvus for Context

Retrieval happens in real-time within the FastAPI backend when a user query is received.

Step 1: Load the Existing Store: When the backend application starts, it initializes a connection to the configured Milvus instance and loads the specified collection into memory for efficient querying, similar to the code shown in section 5.2.

Step 2: Embed User Query: The incoming user query string is converted into an embedding vector using the same Sentence Transformer model that was used during indexing.

```
# In backend application logic
def embed_query(query: str) -> List[float]:
    # text = f"{EMBED_QUERY_PREFIX}{query}" # Apply prefix if needed
    return embedding_model.encode([query])[0].tolist()
```

Step 3: Perform Similarity Search: The `collection.search()` method is used to find the most relevant vectors. This method performs a vector similarity calculation (Inner Product IP is used in this project) between the query vector and all the document vectors in the Milvus collection. It returns the top k results based on the `SEARCH_TOP_K` configuration.

```
# In backend application logic (within the /chat endpoint)
# 1. Embed the user's query
query_vec = embed_query(query)

# 2. Search in Milvus for top K candidates
search_params = {"metric_type": "IP", "params": {"nprobe": 16}} # Example search parameters
results = collection.search(
    data=[query_vec],
    anns_field="embedding", # The field containing vectors
    param=search_params,
    limit=SEARCH_TOP_K,      # How many results to retrieve initially
    output_fields=["chunk_text", "priority", "last_updated_at", "url"], # Metadata to return
)

# 'results' contains a list of hits, each with distance/score and entity data
# Process these results for boosting, reranking, and context generation
hits = []
if results and results[0]:
    for hit in results[0]:
        entity = hit.entity
        raw_score = float(hit.distance)
        # ... extract metadata from entity ...
        hits.append({
            "chunk_text": entity.get("chunk_text"),
            "url": entity.get("url"),
            "raw_score": raw_score,
            # ... other extracted metadata ...
        })
# ... pass 'hits' to boosting and reranking logic ...
```

This list of `matched_chunks` is the "context" that is then passed to the LLM for the generation step.

6. In-Depth Code Explanation

This section provides a detailed walkthrough of the project's source code, focusing on the core logic and architecture.

6.1 Part 1: Data Ingestion Pipeline (The Scrapers)

This offline process is the foundation of the chatbot's knowledge. It transforms unstructured web content into a structured, searchable knowledge base. The new pipeline is a multi-stage process involving crawling, classification, database storage, and vectorization.

The Crawler & Classifier (Replaces web_scraper.py)

- **Purpose:** To build the knowledge base by crawling a sitemap and intelligently classifying each page using an LLM.
- **Core Logic:** The script uses asyncio and the crawl4ai library to fetch pages. For each page, it extracts clean markdown and then makes a *second* LLM call (via the classification logic) to get structured metadata like category, tags, and priority. This structured data is then saved to a central PostgreSQL database.

```
# In the main crawler script
async def crawl_urls(urls, pool, max_pages: int | None = None):
    # ... (setup) ...
    async with AsyncWebCrawler(config=browser_config) as crawler:
        for i, url in enumerate(urls, 1):
            print(f"[{i}/{total}] Crawling: {url}")
            try:
                # 1. Crawl the page and get clean markdown
                results = await crawler.arun(url=url, config=run_config)
            except Exception as e:
                print(f"Crawler error for {url}: {e}")
                continue

            for result in results:
                if result.success:
                    content_md = result.markdown or ""
                    # Basic metadata (URL, checksum, etc.)
                    metadata = extract_metadata(result.html, url, ...)

                    # 2. Classify the content using an LLM
                    async with llm_semaphore:
                        classification = await classify_page(metadata, content_md)
                        metadata["category"] = classification.get("category")
                        metadata["tags"] = classification.get("tags") or []
                        metadata["priority"] = classification.get("priority", ...)
                        llm_cleaned = classification.get("llm_cleaned")

                    # 3. Save to PostgreSQL database
                    upsert_result = await upsert_page(pool, url, metadata, content_md, ...)
                    print(f"Saved and stored: {url}")
```

Data Cleaning & Classification (Replaces Data Cleaning)

- **Explanation:** Instead of manual cleaning with BeautifulSoup, the new process uses two layers. First, the crawler's `excluded_tags` (like header, footer) performs initial structural cleaning. Second, the classification logic uses an LLM to perform *semantic* cleaning and extraction by following a specific prompt.
- **Code (LLM Classification Prompt):** This prompt instructs an LLM to act as the classifier, extracting structured data and producing a final, cleaned version of the text.

```
# In the classification logic script
def build_prompt(categories: List[str], url: str, ...):
    prompt = f"""
You are a helpful classifier. Given a SINGLE webpage... return EXACTLY one JSON object.

Rules:
1. Choose one category. ...
2. Provide a list of tags (short strings) ...
3. Suggest an integer priority from 0 to 100 ...
```

```

4. ...
5. Remove irrelevant sections (navigation menus, headers, footers, cookie notices, ...).
6. Produce a cleaned version of the main text content only.

```

```

Available categories:
{categories_text}

```

```

Input:
url: {url}
...
full_content: {content_snippet}
"""
    return prompt

```

The Database Interface (Replaces scrape_all_packages.py)

- **Purpose:** To store the scraped and classified content in a versioned PostgreSQL database, which acts as the "source of truth".
- **Architecture:** The database logic uses asyncpg for asynchronous database operations. The core function is `upsert_page`, which intelligently handles versioning by comparing a checksum (an SHA256 hash) of the new content against the stored version.
- **Versioning Logic:** This logic is critical. If the content hash is new, a new `page_versions` record is created, and the main `pages` table is updated to point to it. This automatically flags the page (`is_vectorized = FALSE`) for re-indexing by the vectorizer. If the hash is the same, it simply updates metadata.

```

# In the database logic script
async def upsert_page(pool, url: str, metadata: dict, content_md: str, ...):
    async with pool.acquire() as conn:
        async with conn.transaction():
            row = await get_page_by_url(conn, url) # Check if page exists

            if row is None:
                # Case 1: New Page -> Insert new page and page_version
                # ... (INSERT INTO pages...)
                # ... (INSERT INTO page_versions...)
            else:
                # Case 2: Existing Page
                page_id = row["page_id"]
                current_version_id = row["current_version_id"]

                # Get checksum of the currently active version
                current_checksum = (await get_version_by_id(conn,
                    current_version_id))["checksum"]

                if metadata.get("checksum") != current_checksum:
                    # Content Changed -> Create new version
                    version_id = uuid.uuid4()
                    # Set old version is_current = FALSE
                    await conn.execute("UPDATE page_versions SET is_current = FALSE ...")
                    # Insert new version (is_current = TRUE)
                    # ... (INSERT INTO page_versions...)
                    # Update main 'pages' table to point to new version_id
                    await conn.execute("UPDATE pages SET current_version_id = $1, ...")
                else:
                    # Content Unchanged -> Only update metadata
                    # ... (UPDATE pages SET last_scraped_at = now(), ...)

```

The Vectorizer (Replaces Extraction Logic and Data Structuring)

- **Purpose:** This script runs as a *separate* process. Its sole job is to read from the PostgreSQL database and populate the Milvus vector store, keeping the search index in sync with the content database.
- **Core Logic:** It periodically fetches pages where `is_vectorized = FALSE`. It then chunks the content, generates vector embeddings, and upserts them to Milvus. This decouples the compute-intensive embedding process from the crawling process.

```
# In the vectorization script
async def process_one_page(pool, collection, model, row):
    page_id = row["page_id"]
    content_md = row["content_md"] or ""
    version_id = row["current_version_id"] or ""

    # 1. If there are old vectors, delete them first
    delete_vectors_by_page(collection, page_id)

    # 2. Chunk content
    chunks = get_text_chunks(content_md)
    if not chunks:
        # ... (mark as vectorized and skip) ...
        return

    # 3. Embed chunks in batches
    vectors_to_insert = []
    for i in range(0, len(chunks), BATCH_SIZE):
        batch_chunks = chunks[i: i + BATCH_SIZE]
        embeddings = await asyncio.to_thread(embed_texts, model, batch_chunks)

        for j, emb in enumerate(embeddings):
            # ... (append vector data to vectors_to_insert list) ...

    # 4. Upsert new vectors to Milvus
    upsert_vectors(collection, vectors_to_insert)

    # 5. Mark page as vectorized in PostgreSQL
    await mark_page_vectorized(pool, page_id)
```

6.2 Part 2: Core Chatbot Logic & User Interface

Backend Logic (FastAPI)

The FastAPI server is the central nervous system of the chatbot, handling the entire real-time RAG process.

Application Initialization (Replaces SLTChatbot Class and Initialization)

Instead of a single class, the FastAPI application initializes several components at startup. It loads the embedding model and the reranker model into memory and establishes a persistent connection to the Milvus vector database.

```
# In the main backend application script

# 1. Load Embedding and Reranker models
embedding_model = SentenceTransformer(EMBEDDING_MODEL)

reranker: Optional[Any] = None
if ENABLE_RERANK and _HAS_RERANKER:
    try:
        reranker = CrossEncoder(RERANK_MODEL)
    except Exception as e:
        print(f"Reranker init failed: {e}")
        reranker = None

# 2. Connect to and load Milvus Collection
connections.connect(alias="default", uri=URI, token=TOKEN)
collection = Collection(COLLECTION_NAME)
collection.load()

# 3. Initialize FastAPI app
app = FastAPI()
# ... (middleware) ...
```

Query Lifecycle - Step 1: Retrieval (Replaces Classification and Retrieval)

When a query arrives at the `/chat` endpoint, it is *first* embedded. This query vector is then used to search the Milvus collection. This is a semantic search, replacing the old keyword-based classification, and retrieves a large set of candidates.

```
# In the main backend application script
@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(req: ChatRequest):
    query = req.message.strip()

    # ... (update history) ...

    # 1. Embed the user's query
    query_vec = embed_query(query)

    # 2. Search in Milvus for top K candidates
    search_params = {"metric_type": "IP", "params": {"nprobe": 16}}
    results = collection.search(
        data=[query_vec],
        anns_field="embedding",
        param=search_params,
        limit=SEARCH_TOP_K,
        output_fields=["chunk_text", "priority", "last_updated_at", "url"],
    )
    # ... (rest of the steps) ...
```

Query Lifecycle - Step 2: Boosting and Reranking (New, more advanced retrieval logic)

The raw results from Milvus are not immediately sent to the LLM. They go through a two-stage refinement process. First, a `_score_with_boost` function adds small bonuses for priority (important pages) and recency (newer content). Second, these boosted results are passed to a CrossEncoder reranker model, which re-scores them for semantic relevance to the query, providing a much more accurate final context.

```

# In the main backend application script

# ... (inside chat_endpoint, after getting 'results') ...

# 3. Apply boosting
hits = []
for h in results[0]:
    # ... (extract entity data: raw_score, priority, timestamp) ...
    boosted = _score_with_boost(raw_score, pr, ts)
    hits.append({ ... "boosted_score": boosted, ... })

# 4. Rerank the best candidates
# (Deduplication logic is omitted for clarity)
best = _rerank(query, deduped[...], CONTEXT_CHUNKS)

# 5. Generate the answer using only the 'best' chunks
answer_data = generate_answer(query, best, ...)
return ChatResponse(**answer_data)

def _rerank(query: str, candidates: List[Dict[str, Any]], top_n: int) -> List[Dict[str, Any]]:
    if reranker is not None:
        pairs = [(query, c.get("chunk_text", "")) for c in candidates]
        scores = reranker.predict(pairs)
        for c, s in zip(candidates, scores):
            c["rerank_score"] = float(s)
        # Sort by the new, more accurate rerank_score
        candidates.sort(key=lambda x: x.get("rerank_score", 0.0), reverse=True)

    return candidates[:top_n]

```

Query Lifecycle - Step 3: Generation (Replaces Generation (query_llm))

This is the "Generation" stage. The function constructs a detailed, context-rich prompt. It combines the user's query, the conversation history, and the *final, reranked* text chunks. This prompt is then sent to the central `query_llm` function (from the multi-provider LLM logic) to get the final answer.

```

# In the main backend application script
def generate_answer(query: str, context_chunks: List[Dict[str, Any]], history: List[str]) -> Dict[str, str]:
    context_text, sources = _format_context_with_sources(context_chunks)
    history_text = "\n".join([f"- {q}" for q in history[-6:]])

    # Construct the final prompt, explicitly instructing the LLM
    prompt = f"""
You are an expert assistant for Sri Lanka Telecom.
Answer the user's question using ONLY the information from the provided sources. ...
Cite evidence using [Source N] inline where relevant.
...
Conversation (last turns):
{history_text}

User question: {query}

Sources:
{context_text}

Format the output exactly as:
Answer: <your answer here with [Source N] citations>
Follow-up question: <one question>
"""

```

```
# Send this prompt to the configured LLM (Ollama or Groq)
response = query_llm(prompt)

# ... (parse the response) ...
return { "reply": answer, "next_suggestion": suggestion }
```

Frontend Interface (index.html)

The index.html file provides the complete user-facing chat window.

Communication (sendMessage)

This async JavaScript function handles the entire client-side communication loop. It uses the fetch API to send the user's message to the /chat endpoint and manages the UI state by disabling the send button and showing a loading animation for a responsive user experience. The request body now sends a session_id instead of a user_id.

```
// In index.html
async function sendMessage(message = null) {
  const userMessage = message || input.value.trim();
  if (!userMessage) return;

  // ... (addMessage to UI, disable button, show loading) ...

  try {
    const response = await fetch('/chat', { // Assumes API is on the same host
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        message: userMessage,
        session_id: 'web_user' // Or a generated session ID
      })
    });
    const data = await response.json();

    // ... (hide loading, add bot 'reply' to UI) ...

  } catch (error) {
    // ... (handle network errors) ...
  } finally {
    // ... (re-enable button) ...
  }
}
```

Dynamic and Interactive UI (addLocationRequestButtons & shareLocation)

This feature, which dynamically added buttons to request the user's location, is **deprecated** in the current architecture. The backend no longer requests location data, and the POST /location endpoint has been removed to streamline the chatbot's focus on RAG-based answers. The frontend logic for this has been removed.

7. Installation and Configuration Guide

7.1 Prerequisites

- Python 3.8+
- Git
- Access to a PostgreSQL database instance.
- Ollama (and a pulled model like `gemma2:2b`)

7.2 `requirements.txt`

```
geopy
tqdm
# pyteseract
beautifulsoup4
aiohttp
# Pillow
asynpg
crawl4ai
pydantic
groq
#torch
setuptools<81

# core
python-dotenv==1.0.1
requests>=2.31.0

fastapi==0.115.0
uvicorn[standard]==0.30.6

# embeddings + reranker
# Use FlagEmbedding for bge-m3 + reranker;
sentence-transformers for utilities
FlagEmbedding==1.2.11
sentence-transformers
transformers

# tokenization / utils
# numpy
scikit-learn>=1.4.2

# vectorize pipeline
pymilvus==2.5.3
langchain # for the text splitters
# uvloop
```

7.3 Step-by-Step Installation

1. **Clone the Repository:** `git clone <repository_url>`
2. **Create Virtual Environment:** `python -m venv venv && source venv/bin/activate`
3. **Install Dependencies:** `pip install -r requirements.txt`
4. **Install Local LLM (Optional):** If you plan to use a local LLM, install [Ollama](#) and pull a model (e.g., `ollama pull mistral`, which is the default in the configuration).

7.4 Data Preparation (Crucial First Step)

Before running the app, you must build the knowledge bases in this order:

1. **Initialize the Database:** Run the database initialization script (e.g., `python db_init.py`). This ensures the database specified in `DATABASE_URL` exists.
2. **Crawl and Populate PostgreSQL:** Run the main crawler script (e.g., `python crawl.py`). This will scrape the website, classify the content, and store it in your primary database.
3. **Vectorize Data into Milvus:** Run the vectorization script (e.g., `python vectorize.py`). This will read the data from PostgreSQL and populate your Milvus collection.

7.5 Configuration

The application is configured via a `.env` file. Create this file in the root directory.

- **Database Configuration:**
 - `DATABASE_URL`: The connection string for your PostgreSQL database.
- **Zilliz Cloud / Milvus Configuration:**
 - `ZILLIZ_CLOUD_URI`: The URI for your Milvus/Zilliz Cloud instance.
 - `ZILLIZ_CLOUD_API_KEY`: The API key for your Milvus instance.
 - `VECTOR_COLLECTION`: The name of the collection in Milvus.
- **Embedding Model Configuration:**
 - `EMBEDDING_MODEL`: The Sentence Transformer model to use.
 - `EMBEDDING_DIM`: The dimension of the chosen model (e.g., 1024 for BAAI/bge-m3).
- **Retrieval Configuration:**
 - `SEARCH_TOP_K`: Number of results to retrieve before reranking.
 - `CONTEXT_CHUNKS`: Number of chunks to send to the LLM.
 - `ENABLE_RERANK`: Set to true to enable the Cross-Encoder reranking step.
 - `RERANK_MODEL`: The model to use for reranking.
- **LLM Provider Configuration:**
 - `GROQ_API_KEY`: Your API key from Groq, used as the cloud provider.
- **Configuring a Local LLM Provider (Optional):**
 - To use a local LLM like Ollama by default, you must edit the `DEFAULT_CONFIG` dictionary in the central LLM logic script.
 - Change `"provider": "groq"` to `"provider": "ollama"`.
 - Ensure the `"ollama_model"` value matches the model you pulled (e.g., `"mistral"`).

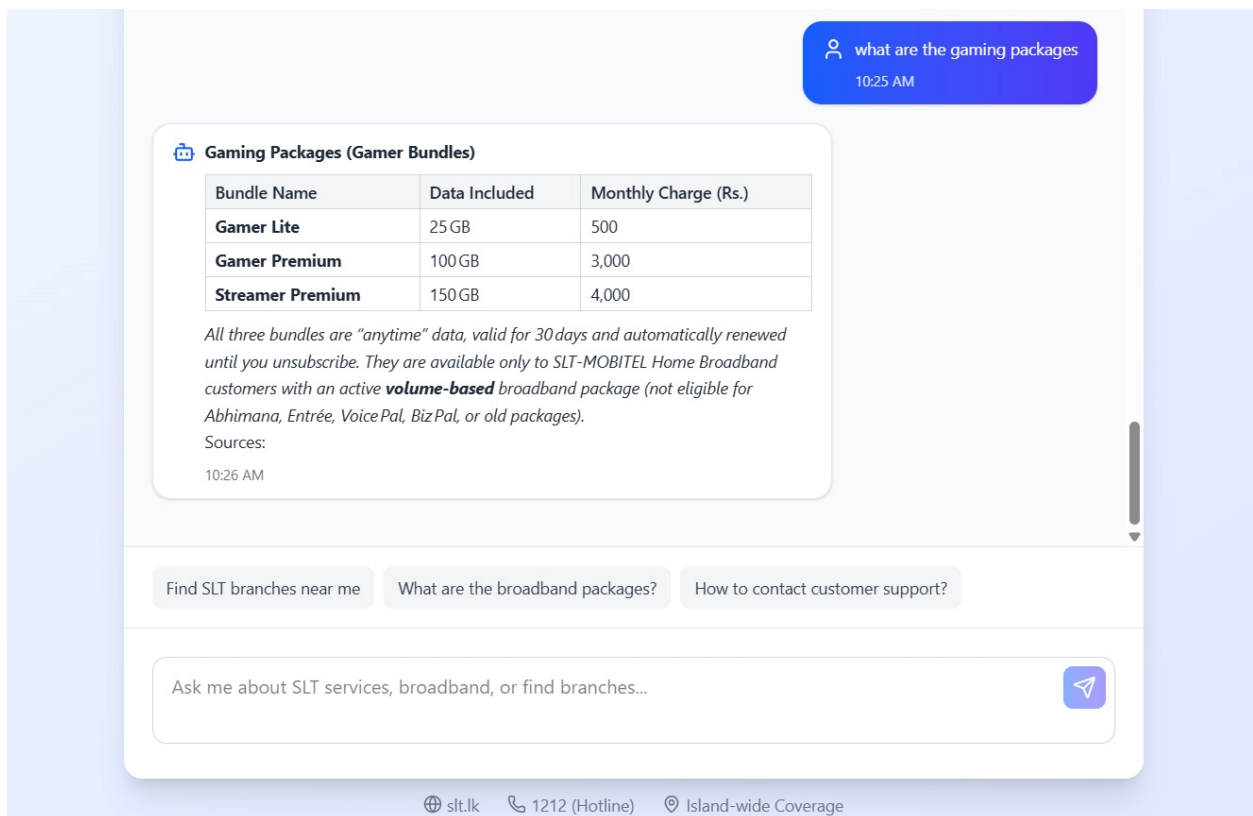
8. User Guide

8.1 Launching the Application

1. Ensure your PostgreSQL and Milvus databases are accessible and configured in .env.
2. **(Optional)** If using a local LLM, ensure your Ollama or LM Studio application is running in the background.
3. Start the FastAPI backend server: `uvicorn app:app --host 0.0.0.0 --port 5000`
4. Open the index.html file (frontend) in a web browser.

8.2 Interacting with the Chatbot

1. **Text Input:** Type your question in the input box at the bottom and press Enter or click "Send".
2. **Quick Actions:** Use the buttons at the top for common queries like "Broadband Packages".



8.3 Example Conversations

- **User:** What are the prepaid packages?
- **User:** Tell me about PEO TV
- **User:** Find branches in Kandy

9. API Reference

The backend exposes several endpoints. The base URL is `http://localhost:4321`.

9.1 `POST /chat`

Handles a user's chat message.

- **Request Body:**

```
{
  "message": "What are the latest fiber packages?",
  "user_id": "web_user_123"
}
```

- **Success Response (200):**

```
{
  "reply": "Here are the latest fiber packages I found...",
  "llm_used": "Local LLM (Ollama)",
  "request_location": false
}
```

9.2 `POST /location`

Processes coordinates to find nearby branches.

- **Request Body:**

```
• {
•   "latitude": 6.9271,
•   "longitude": 79.8612,
•   "user_id": "web_user_123"
• }
```

- **Success Response (200):**

```
• {
•   "reply": "📍 Using your current location...\n🏠 Nearest SLT  
Branches:\n..."
• }
```

10. Testing Strategy

- **Unit Testing:** Using pytest to test individual functions like `_score_with_boost` in the backend logic or data validation functions in the database interface.
- **Integration Testing:** Testing the interaction between the backend application and the PostgreSQL and Milvus databases to ensure queries and data flows work correctly.
- **End-to-End (E2E) Testing:** Using a tool like Playwright to script a full user journey: opening the frontend HTML file, typing a message, clicking send, and asserting that the expected response appears in the UI.

11. Deployment

- **Production Environment Setup:** The FastAPI development server (`uvicorn`) is not suitable for production. A production-ready setup would involve an ASGI server like Gunicorn.
- **Deployment with Gunicorn and Nginx:**
 - Install Gunicorn: `pip install gunicorn uvicorn`.
 - Run the app with Gunicorn: `gunicorn -w 4 -k uvicorn.workers.UvicornWorker app:app --bind 0.0.0.0:8000`.
 - Use Nginx as a reverse proxy to forward requests to Gunicorn, handle SSL, and serve static files.

12. Security Considerations

- **API Security:** The current API is open. In a production environment, consider adding API keys or rate limiting to prevent abuse.
- **Credential Management:** Ensure database and Milvus credentials (`DATABASE_URL`, `ZILLIZ_CLOUD_API_KEY`, `GROQ_API_KEY`) are managed securely using environment variables, not hardcoded.
- **Data Privacy:** Using Ollama keeps all data local. If using Groq, user queries and retrieved context are sent to Groq's servers, which should be disclosed in a privacy policy.
- **Dependency Management:** Regularly scan dependencies for known vulnerabilities using a tool like `pip-audit`.

13. Maintenance and Operations

- **Monitoring and Logging:** The application uses Python's `logging` module. In production, logs should be aggregated to a central location (e.g., ELK stack, Datadog) for monitoring and alerting.
- **Updating the Knowledge Base:** To update the chatbot's knowledge, the data pipeline scripts must be re-run. This process should be automated to run on a schedule (e.g., weekly) using a cron job or a CI/CD pipeline. The flow would be to run the crawler script followed by the vectorization script.

14. Troubleshooting

Issue	Possible Cause & Solution
Connection Error on Frontend	The FastAPI backend is not running. Start it with <code>uvicorn app:app</code> .
LLM Fails to Respond	If using Groq, the <code>GROQ_API_KEY</code> may be invalid or missing. If using Ollama, ensure the Ollama service is running and the model (e.g., <code>mistral</code>) has been pulled.
"Vector Store Not Found" / Milvus Error	- The vectorization script has not been run. - Environment variables for Milvus are incorrect. - Network issue connecting to Zilliz Cloud.
PostgreSQL Connection Error	- The <code>DATABASE_URL</code> environment variable is incorrect. - The PostgreSQL server is down or inaccessible. - The database initialization script was not run to create the database.
Scraper Fails	The target website's sitemap or HTML structure may have changed. The crawler script may need to be updated.
Slow Responses (Ollama)	The host machine may not have a dedicated GPU or enough RAM. Response time is hardware-dependent for local LLMs.

15. Future Work and Improvements

- **Enhanced Session Management:** Implement a more robust session management system (e.g., using Redis) to handle a larger number of concurrent users and longer conversation histories.
- **Admin Dashboard:** Create a dashboard to monitor conversations, view performance metrics, manually update knowledge sources, and retrain the data pipeline.
- **CI/CD Pipeline:** Automate the process of scraping and updating the vector database on a regular schedule using tools like GitHub Actions or Jenkins.
- **Expand Knowledge Base:** Scrape additional SLT services like mobile plans and enterprise solutions to broaden the chatbot's capabilities.